



# ES6 Arrow Function (Part 1)

---

*Beginner → Advanced Guide for Software Development*



## Table of Contents

- What is Arrow Function?
- Why Learn Arrow Function?
- Prerequisites
- Theory
- Normal Function vs Arrow Function
- Syntax
- Rules
- Basic Examples
- Line by Line Explanation
- Real Project Example
- Benefits
- Common Mistakes
- Best Practices
- Interview Questions
- Practice



## What is Arrow Function?

---

### English

Arrow Function is a modern JavaScript function syntax introduced in **ES6 (ECMAScript 2015)**.

It provides a shorter, cleaner, and more readable way to write functions.

Arrow Functions are heavily used in **React**, **Node.js**, **Next.js**, **Express.js**, and modern JavaScript applications.

## বাংলা

Arrow Function হলো JavaScript-এর একটি আধুনিক Function Syntax যা ES6 (ECMAScript 2015)-এ যোগ করা হয়েছে।

এটি Function লেখাকে ছোট, পরিষ্কার এবং সহজ করে।

বর্তমান সময়ে React, Node.js, Next.js, Express এবং প্রায় সব Modern JavaScript Project-এ Arrow Function ব্যবহার করা হয়।

সহজ ভাষায়,

আগে Function লেখতে অনেক Code লিখতে হতো।

ES6-এর পরে Arrow Function আসায় একই কাজ অনেক কম Code-এ করা যায়।

Example

Normal Function

```
function greet() {  
  console.log("Hello");  
}
```

Arrow Function

```
const greet = () => {  
  console.log("Hello");  
};
```

দুইটিই একই কাজ করে।

কিন্তু Arrow Function বেশি Modern এবং Readable।

## Why Learn Arrow Function?

React শেখার আগে Arrow Function অবশ্যই জানতে হবে।

কারণ React-এর প্রায় প্রতিটি Component, Event Handler, Callback Function এবং Array Method-এ Arrow Function ব্যবহার করা হয়।

React Example

```
const Home = () => {  
  return <h1>Home Page</h1>;  
};
```

Event Example

```
<button onClick={() => console.log("Clicked")}>  
  Click  
</button>
```

Array Example

```
users.map(user => console.log(user));
```

তুমি যদি Arrow Function না জানো,

তাহলে React-এর অনেক Code বুঝতে সমস্যা হবে।

## Prerequisites

---

Arrow Function শেখার আগে নিচের বিষয়গুলো জানা থাকলে সহজ হবে।

- Variable
- Function
- Parameters
- Return
- const
- let

## Theory

---

ধরো তোমার কাছে একটি Calculator আছে।

Calculator-এ Add, Subtract, Multiply, Divide নামে অনেক Function আছে।

আগে Function লিখতে এমন হতো।

```
function add(a, b) {  
  return a + b;  
}
```

ES6-এর পরে JavaScript বললো,

"এত বড় করে লেখার দরকার নেই।"

একই Function আরও ছোট করে লেখা যাবে।

```
const add = (a, b) => {  
  return a + b;  
};
```

এটাই Arrow Function।

অর্থাৎ,

Arrow Function নতুন কোনো Function নয়।

এটি Function লেখার নতুন Syntax।

## Normal Function vs Arrow Function

---

Normal Function

```
function greet(name) {  
  return "Hello " + name;  
}
```

Arrow Function

```
const greet = (name) => {  
  return "Hello " + name;  
};
```

দুইটির Output একই।

Difference শুধু Syntax।

## Basic Syntax

---

```
const functionName = (parameters) => {  
  // Code  
};
```

Syntax Breakdown

```
const add = (a, b) => {  
  return a + b;  
};
```

এখানে,

**const** → একটি Variable তৈরি করছে।

**add** → Function-এর নাম।

**(a, b)** → Parameters।

**=>** → Arrow Operator।

**{}** → Function Body।

**return** → Result ফেরত পাঠাচ্ছে।

## Rules

---

### Rule 1

Arrow Function সাধারণত const দিয়ে তৈরি করা হয়।

```
const sayHello = () => {};
```

## Rule 2

Function Keyword ব্যবহার করা হয় না।

Wrong

```
function () => {}
```

Correct

```
const demo = () => {};
```

## Rule 3

Arrow ( `=>` ) অবশ্যই Parameter-এর পরে বসবে।

Correct

```
const sum = (a, b) => {};
```

Wrong

```
const sum => (a, b) {};
```



## Example 1

```
const greet = () => {  
  console.log("Hello World");  
};  
  
greet();
```

Output

```
Hello World
```

## Line by Line Explanation

```
const greet = () => {
```

Explanation

**const**

→ greet নামে Variable তৈরি করছে।

**greet**

→ Function-এর নাম।

**()**

→ কোনো Parameter নেই।

**=>**

→ Arrow Function শুরু।

**{**

→ Function Body শুরু।

```
console.log("Hello World");
```

Explanation

Console-এ Hello World দেখাবে।

```
};
```

Explanation

Function শেষ।

```
greet();
```

Explanation

Function Call করা হয়েছে।

Call না করলে Function Execute হবে না।



## Example 2

---

```
const printName = () => {  
  console.log("Rokon");  
};  
  
printName();
```

Output

```
Rokon
```



## Example 3

---

```
const welcome = () => {  
  console.log("Welcome to JavaScript");  
};  
  
welcome();
```

Output



Welcome to JavaScript



## Real Project Example

---

ধরো তুমি React দিয়ে একটি Portfolio Website বানাচ্ছে।

Home.jsx

```
const Home = () => {  
  
  return (  
  
    <div>  
  
      <h1>Welcome</h1>  
  
    </div>  
  
  );  
  
};  
  
export default Home;
```

এখানে,

```
const Home = () => {}
```

এটাই Arrow Function।

React-এর প্রায় প্রতিটি Component এই Style-এ লেখা হয়।

আরেকটি Example

Navbar.jsx

```
const Navbar = () => {
```

```
return (  
  
  <nav>  
  
    Navbar  
  
  </nav>  
  
);  
  
};  
  
export default Navbar;
```

Professional React Project-এর প্রায় সব Component এভাবেই তৈরি করা হয়।

## Benefits

---

- কম Code লিখতে হয়।
- Code Readable হয়।
- Modern JavaScript Standard।
- React-এর সাথে Perfect।
- Callback Function লিখতে সহজ।
- Array Method-এর সাথে বেশি ব্যবহার হয়।
- Clean Code লিখতে সাহায্য করে।

## Common Mistakes

---

### 1. function Keyword ব্যবহার করা

Wrong

```
function => {}
```

Correct

```
const demo = () => {};
```

## 2. Arrow ভুল জায়গায় লেখা

Wrong

```
const add => (a,b) {}
```

Correct

```
const add = (a,b) => {};
```

## 3. Function Call না করা

```
const hello = () => {  
  console.log("Hello");  
};
```

এটি শুধু Function তৈরি করেছে।

Execute করতে হবে।

```
hello();
```



## Best Practices

- Function-এর জন্য Meaningful Name ব্যবহার করো।

- Arrow Function-এর জন্য `const` ব্যবহার করো।
- Code ছোট এবং পরিষ্কার রাখো।
- React Component সবসময় Arrow Function দিয়ে লেখার অভ্যাস করো।
- একই Project-এ Function Style একরকম রাখো।

## Interview Questions

---

### Arrow Function কী?

### ES6-এ কেন Arrow Function যোগ করা হয়েছে?

### Normal Function এবং Arrow Function-এর পার্থক্য কী?

### Arrow Function-এর Syntax কী?

### React-এ Arrow Function বেশি ব্যবহার করা হয় কেন?



## Practice

---

### Practice 1

একটি Arrow Function তৈরি করো।

Function Name

```
hello
```

Output

```
Hello JavaScript
```

### Practice 2

একটি Arrow Function তৈরি করো।

Function Name

```
developer
```

Output

```
I Want To Be A Software Engineer
```

### Practice 3

Portfolio Website-এর জন্য

```
Home
```

```
About
```

```
Contact
```

নামে তিনটি Arrow Function Component তৈরি করো।



## Summary

---

Arrow Function হলো ES6-এর একটি Modern Function Syntax।

এটি Code-কে ছোট, পরিষ্কার এবং Readable করে।

React, Node.js, Express, Next.js এবং Modern JavaScript Development-এ Arrow Function সবচেয়ে বেশি ব্যবহৃত হয়।

Arrow Function ভালোভাবে শিখে ফেললে React-এর প্রায় প্রতিটি Component এবং Callback Function সহজে বুঝতে পারবে।



## ES6 Arrow Function (Part 2)

---

## Table of Contents

- Parameters
- No Parameter
- Single Parameter
- Multiple Parameters
- Default Parameters
- Return Statement
- Implicit Return
- Explicit Return
- Returning Objects
- Returning Arrays
- Real Project Examples
- Common Mistakes
- Best Practices
- Interview Questions
- Practice

## Parameters

---

### English

A parameter is a variable that receives data when a function is called.

### বাংলা

Parameter হলো Function-এর ভিতরের একটি Variable যা Function Call করার সময় Value গ্রহণ করে।

Example

```
const greet = (name) => {  
  console.log(name);  
};  
  
greet("Rokon");
```

এখানে,

**name**

হলো Parameter।

আর

**"Rokon"**

হলো Argument।

মনে রাখবে,

Parameter → Function তৈরি করার সময়

Argument → Function Call করার সময়



## No Parameter

---

যদি Function-এর কোনো Data প্রয়োজন না হয়,

তাহলে Empty Parentheses ব্যবহার করতে হবে।

```
const hello = () => {  
  console.log("Hello World");  
};  
  
hello();
```

Output

```
Hello World
```

## Explanation

()

মানে Function কোনো Parameter নিচ্ছে না।



## Single Parameter

---

যদি মাত্র একটি Parameter থাকে,

তাহলে Parentheses দেওয়া Optional।

দুইটিই সঠিক।

```
const square = (num) => {  
  return num * num;  
};
```

```
const square = num => {  
  return num * num;  
};
```

Output

```
25
```

## Best Practice

React Project-এ Parentheses ব্যবহার করলে Code বেশি Readable হয়।

Recommended



```
const square = (num) => {};
```



## Multiple Parameters

---

একাধিক Parameter থাকলে Parentheses বাধ্যতামূলক।

```
const add = (a, b) => {  
  return a + b;  
};  
  
console.log(add(10, 20));
```

Output

```
30
```

Explanation

```
a = 10  
  
b = 20
```

তারপর

```
10 + 20
```

Return করবে।



## Default Parameters

---

যদি User কোনো Value না পাঠায়,

তাহলে Default Value ব্যবহার করা যায়।

```
const greet = (name = "Guest") => {  
  console.log(`Hello ${name}`);  
};  
  
greet();  
  
greet("Rokon");
```

Output

```
Hello Guest  
  
Hello Rokon
```

কেন দরকার?

অনেক সময় User Input দেয় না।

Default Parameter Application Crash হওয়া থেকে বাঁচায়।

## Return Statement

---

Return Function-এর Result বাইরে পাঠায়।

Example

```
const add = (a, b) => {  
  return a + b;  
};  
  
const result = add(5, 6);  
  
console.log(result);
```

Output

11

## Without Return

```
const add = (a, b) => {  
  a + b;  
};  
  
console.log(add(5,6));
```

Output

undefined

কারণ,

কিছু Return করা হয়নি।



## Explicit Return

---

যখন Curly Braces ব্যবহার করা হয়,

তখন Return Keyword লিখতে হবে।

```
const multiply = (a, b) => {  
  return a * b;  
};
```

এটিকে Explicit Return বলে।



## Implicit Return

---

যদি Function-এর Body-তে মাত্র একটি Expression থাকে,  
তাহলে Return Keyword লিখতে হয় না।

```
const multiply = (a, b) => a * b;
```

Output

```
30
```

React-এ এটি খুব বেশি ব্যবহার হয়।

Example

```
const fullName = (first, last) => `${first} ${last}`;
```

Output

```
Rokon Uzzaman
```



## Returning Objects

---

Arrow Function-এ Object Return করার সময়

Parentheses ব্যবহার করতে হয়।

Wrong

```
const user = () => {  
  name: "Rokon"  
};
```

কারণ JavaScript এটিকে Object হিসেবে বুঝবে না।

Correct

```
const user = () => ({  
  name: "Rokon",  
  age: 23  
});
```

Output

```
{  
  name: "Rokon",  
  age: 23  
}
```

কেন Parentheses?

কারণ,

()

JavaScript-কে বলে,

"এটি Object Literal"



## Returning Arrays

Array Return করা খুব সহজ।

```
const numbers = () => [10, 20, 30, 40];
```

Output

```
[10, 20, 30, 40]
```



## Real Project Example 1

---

ধরো,

তুমি একটি E-Commerce Website বানাচ্ছে।

Product Price Calculate

```
const calculatePrice = (price, quantity) => price * quantity;  
  
console.log(calculatePrice(500,2));
```

Output

```
1000
```

React Project-এ এই ধরনের Utility Function অনেক ব্যবহার হয়।



## Real Project Example 2

---

User Full Name

```
const getFullName = (firstName,lastName) =>  
`${firstName} ${lastName}`;  
  
console.log(getFullName("Rokon", "Uzzaman"));
```

Output

```
Rokon Uzzaman
```



## Real Project Example 3

---

React Component

```
const Hero = () => {  
  
  return (  
  
    <section>  
  
      <h1>Welcome</h1>  
  
    </section>  
  
  );  
  
};  
  
export default Hero;
```

React-এর প্রতিটি Component মূলত Arrow Function।

## Common Mistakes

---

### Forgetting Return

Wrong

```
const add = (a,b)=>{  
  
  a+b;  
  
}
```

Correct

```
const add = (a,b)=>{  
  
  return a+b;  
  
}
```

অথবা

```
const add = (a,b)=>a+b;
```

## Returning Object Incorrectly

Wrong

```
const person = () => {  
  name: "Rokon"  
};
```

Correct

```
const person = () => ({  
  name: "Rokon"  
});
```

## Missing Parentheses

Wrong

```
const sum = a,b => a+b;
```

Correct



```
const sum = (a,b) => a+b;
```

## Best Practices

---

- সবসময় Meaningful Parameter Name ব্যবহার করো।
- Single Line হলে Implicit Return ব্যবহার করো।
- Complex Logic হলে Explicit Return ব্যবহার করো।
- React Component-এর জন্য Arrow Function ব্যবহার করো।
- Utility Function ছোট রাখো।

## Interview Questions

---

**Parameter এবং Argument-এর পার্থক্য কী?**

**Implicit Return কী?**

**Explicit Return কী?**

**Arrow Function থেকে Object Return করতে Parentheses কেন লাগে?**

**Arrow Function-এ Parentheses কখন Optional?**

**Default Parameter কেন ব্যবহার করা হয়?**



## Practice

---

### Beginner

একটি Arrow Function তৈরি করো।

Function Name

```
cube
```

Output

```
125
```

## Intermediate

একটি Arrow Function লিখো।

Input

```
price  
quantity
```

Output

```
Total Price
```

## Advanced

একটি Arrow Function লিখো

যা একটি Student Object Return করবে।

Properties

```
name  
age
```

## Summary

---

এই Part-এ তুমি শিখলে,

- Parameter
- Argument
- No Parameter
- Single Parameter
- Multiple Parameters
- Default Parameter
- Return
- Implicit Return
- Explicit Return
- Returning Object
- Returning Array

এগুলো React Development-এ প্রতিদিন ব্যবহার হয়।

## ES6 Arrow Function (Part 3A)

---

*Beginner → Advanced Guide for Software Development*

## Table of Contents

- Callback Function
- Why Callback Function?
- Normal Function vs Callback Function
- Callback with Arrow Function
- forEach()
- map()

- React List Rendering
- Line by Line Explanation
- Real Project Example
- Common Mistakes
- Best Practices
- Interview Questions
- Practice



## What is a Callback Function?

---

### English

A Callback Function is a function that is passed as an argument to another function and is executed later.

### বাংলা

Callback Function হলো এমন একটি Function যাকে অন্য একটি Function-এর ভিতরে Argument হিসেবে পাঠানো হয়।

অন্য Function যখন প্রয়োজন মনে করে, তখন Callback Function-কে Execute করে।

সহজ ভাষায়,

ধরো তুমি একটি Restaurant-এ খাবার Order করলে।

তুমি Waiter-কে বললে,

"খাবার তৈরি হলে আমাকে ডাকবেন।"

তুমি নিজে বারবার Kitchen-এ যাচ্ছে না।

Kitchen প্রস্তুত হলে Waiter তোমাকে ডাকবে।

JavaScript-এও ঠিক একই ব্যাপার।

তুমি একটি Function অন্য Function-এর কাছে রেখে দাও।

যখন দরকার হবে,

সেটি Call করবে।

এ কারণেই এর নাম Callback Function।



## Why Learn Callback Function?

---

React-এর প্রায় সব জায়গায় Callback Function ব্যবহার হয়।

যেমন

- map()
- filter()
- find()
- reduce()
- Event Handling
- onClick
- useEffect
- Promise
- setTimeout()
- Fetch API

Callback না জানলে React বোঝা কঠিন হয়ে যাবে।



## Normal Function vs Callback Function

---

Normal Function

```
const greet = () => {  
  console.log("Hello");  
};  
  
greet();
```

এখানে Function নিজেই Call হয়েছে।

Callback Function

```
const greet = () => {  
  console.log("Hello");  
};
```

```
setTimeout(greet, 2000);
```

এখানে

```
greet
```

Function-কে Execute করা হয়নি।

Function-টিকে Argument হিসেবে পাঠানো হয়েছে।

২ সেকেন্ড পরে JavaScript নিজে Call করবে।



## Callback Example 1

---

```
const welcome = () => {  
  console.log("Welcome");  
};  
  
const execute = (callback) => {  
  callback();  
};  
  
execute(welcome);
```

Output

```
Welcome
```



## Line by Line Explanation

---

```
const welcome = () => {
```

Explanation

Arrow Function তৈরি হয়েছে।

```
const execute = (callback) => {
```

Explanation

এখানে callback একটি Parameter।

কিন্তু এটি সাধারণ Variable নয়।

এটি একটি Function গ্রহণ করবে।

```
callback();
```

Explanation

Function Call করা হচ্ছে।

```
execute(welcome);
```

Explanation

welcome Function-কে Argument হিসেবে পাঠানো হয়েছে।

execute() Function,

welcome Function-কে Call করেছে।



## Callback Example 2

---

```
const success = () => {  
  console.log("Login Successful");  
};  
  
const login = (callback) => {  
  console.log("Checking User...");
```

```
        callback();  
    };  
  
    login(success);
```

Output

```
Checking User...  
  
Login Successful
```

বাস্তব Project-এ Login Success হলে Callback ব্যবহার করা হয়।

## **forEach()**

---

### English

forEach() executes a callback function once for every array element.

### বাংলা

forEach()

Array-এর প্রতিটি Element-এর জন্য একবার করে Callback Function চালায়।

Syntax

```
array.forEach((item) => {  
  
});
```

Example

```
const fruits = [  
  
    "Apple",
```



```
    "Orange",  
  
    "Banana"  
  ];  
  
  fruits.forEach((fruit) => {  
  
    console.log(fruit);  
  
  });
```

Output

```
Apple  
  
Orange  
  
Banana
```

## Line by Line

```
fruits.forEach()
```

Array-এর প্রতিটি Item-এর উপর Loop চালাবে।

```
(fruit)
```

বর্তমান Element।

```
console.log(fruit)
```

বর্তমান Element Print করবে।

## map()

---

### English

map() creates a new array by transforming every element.

### বাংলা

map()

পুরনো Array পরিবর্তন করে না।

বরং একটি নতুন Array তৈরি করে।

এটি React-এর সবচেয়ে বেশি ব্যবহৃত Array Method।

Syntax

```
array.map((item) => {  
  
});
```



### Example 1

---

```
const numbers = [1,2,3,4];  
  
const doubled = numbers.map((number) => {  
  
    return number * 2;  
  
});  
  
console.log(doubled);
```

Output

```
[2,4,6,8]
```

Original Array

```
[1,2,3,4]
```

New Array

```
[2,4,6,8]
```

Original Array পরিবর্তন হয়নি।



## Implicit Return

---

```
const numbers = [1,2,3];  
  
const result = numbers.map(number => number * 2);
```

এটি React-এ সবচেয়ে বেশি দেখা যায়।



## Line by Line Explanation

---

```
numbers.map()
```

Array-এর প্রতিটি Element-এর উপর Loop চালাবে।

```
(number)
```

বর্তমান Number।

```
number * 2
```

প্রতিটি Number দ্বিগুণ করবে।

map()

সবগুলো Result নিয়ে নতুন Array তৈরি করবে।



## React Example

---

ধরো Database থেকে Users এসেছে।

```
const users = [  
  
  "Rokon",  
  
  "Sakib",  
  
  "Rakib"  
  
];
```

React

```
function App() {  
  
  return (  
  
    <div>  
  
      {  
  
        users.map((user) => (  
  
          <h2>{user}</h2>  
  
        ))  
  
      }  
  
    )  
  
  }  
  
}
```

```
        </div>
      );
    }
  }
```

Output

```
Rokon
Sakib
Rakib
```

React-এ List Render করার জন্য

```
map()
```

সবচেয়ে বেশি ব্যবহার করা হয়।



## Real Project Example

---

ধরো,

তুমি একটি E-Commerce Website বানাচ্ছে।

Products

```
const products = [
  {
    id:1,
    name:"Laptop"
  },

```

```
{  
  id:2,  
  name:"Mouse"  
},  
  
{  
  id:3,  
  name:"Keyboard"  
}  
];
```

React

```
function Products() {  
  return (  
    <div>  
      {  
        products.map((product)=>(  
          <h2 key={product.id}>  
            {product.name}  
          </h2>  
        ))  
      }  
    </div>  
  );  
}
```

Output

```
Laptop  
Mouse  
Keyboard
```

এটি Professional React Project-এর Standard Pattern।

## Common Mistakes

---

### Forgetting Return

Wrong

```
const numbers = [1,2,3];  
  
numbers.map((num)=>{  
  num*2;  
});
```

Output

```
undefined
```

Correct

```
numbers.map((num)=>{  
  
  return num*2;  
  
});
```

অথবা

```
numbers.map(num=>num*2);
```

## Using forEach Instead of map()

Wrong

```
const result = numbers.forEach(num=>num*2);
```

forEach()

কোনো নতুন Array Return করে না।

map()

নতুন Array Return করে।

## Best Practices

---

- Data Transform করতে map() ব্যবহার করো।
- শুধু Loop করার জন্য forEach() ব্যবহার করো।
- React List Render করতে সবসময় map() ব্যবহার করো।
- JSX-তে map() ব্যবহার করলে প্রতিটি Element-এ unique **key** দাও।
- Callback Function-এর নাম Meaningful রাখো।

## Interview Questions

---



## Callback Function কী?

Callback Function কেন ব্যবহার করা হয়?

map() এবং forEach() এর পার্থক্য কী?

React-এ map() এত বেশি ব্যবহার হয় কেন?

map() কি Original Array পরিবর্তন করে?

forEach() কি Return করে?



## Practice

---

### Beginner

একটি Array তৈরি করো।

```
10
```

```
20
```

```
30
```

```
40
```

map()

ব্যবহার করে প্রতিটি Number-কে ৫ দিয়ে গুণ করো।

### Intermediate

একটি Student Array তৈরি করো।

map()

ব্যবহার করে শুধু Student Name Print করো।

## Advanced

React-এ একটি Product Card List তৈরি করো।

Database

```
id  
  
name  
  
price  
  
image
```

map()

ব্যবহার করে সব Product Render করো।



## Summary

---

এই Part-এ তুমি শিখলে

- Callback Function
- Callback Theory
- Callback Example
- forEach()
- map()
- React List Rendering
- Real Project Example
- Common Mistakes
- Best Practices

এগুলো React Development-এর সবচেয়ে গুরুত্বপূর্ণ Concepts। `map()` এবং Callback Function ভালোভাবে আয়ত্ত করতে পারলে Dynamic UI তৈরি করা অনেক সহজ হয়ে যাবে।



## ES6 Arrow Function (Part 3B-1)

---

## Table of Contents

- filter()
- Why filter()?
- filter() Syntax
- filter() Examples
- Line by Line Explanation
- find()
- Why find()?
- find() Syntax
- find() Examples
- findIndex()
- React Examples
- Real Project Example
- Common Mistakes
- Best Practices
- Interview Questions
- Practice

## filter()

---

### English

The **filter()** method creates a **new array** containing only the elements that satisfy a given condition.

### বাংলা

**filter()** এমন একটি Array Method যা নির্দিষ্ট Condition অনুযায়ী Element বাছাই করে একটি **নতুন Array** তৈরি করে।

Original Array কখনো পরিবর্তন করে না।

সহজ ভাষায়,

ধরো একটি কলেজে ১০০ জন Student আছে।

তুমি শুধু যাদের GPA 3.50 বা তার বেশি তাদের List চাও।

সব Student লাগবে না।

শুধু নির্দিষ্ট Condition পূরণ করা Student লাগবে।

এই কাজটাই `filter()` করে।

## Why Use filter()?

---

Software Development-এ `filter()` অনেক বেশি ব্যবহার হয়।

যেমন

- Active Users দেখানো
- Available Products দেখানো
- Completed Tasks
- Paid Orders
- Search Result
- Category অনুযায়ী Product দেখানো
- Admin Panel Filter

React Project-এ API থেকে Data আসার পর Filter করা খুবই সাধারণ কাজ।

## Syntax

---

```
array.filter((item) => {  
  
  return condition;  
  
});
```

Short Syntax

```
array.filter(item => condition);
```



## Example 1

---

```
const numbers = [10,20,30,40,50];

const result = numbers.filter((number)=>{

    return number > 25;

});

console.log(result);
```

Output

```
[30,40,50]
```



## Line by Line Explanation

---

```
numbers.filter()
```

Array-এর প্রতিটি Element-এর উপর Loop চালাবে।

```
(number)
```

বর্তমান Element।

```
return number > 25;
```

যদি Condition সত্য হয়

Element নতুন Array-এ যাবে।

যদি False হয়

Element বাদ যাবে।

Original Array

```
[10, 20, 30, 40, 50]
```

New Array

```
[30, 40, 50]
```



## Example 2

---

Even Numbers

```
const numbers = [1, 2, 3, 4, 5, 6];  
  
const even = numbers.filter(num => num % 2 === 0);  
  
console.log(even);
```

Output

```
[2, 4, 6]
```



## Example 3

---

Student GPA

```
const students = [  
  {  
    name: "Rokon",  
    cgpa: 3.80  
  },  
  {  
    name: "Sakib",  
    cgpa: 3.10  
  },  
  {  
    name: "Rakib",  
    cgpa: 3.65  
  }  
];  
  
const topper = students.filter(student => student.cgpa >= 3.50);  
  
console.log(topper);
```

Output

```
[
  {name:"Rokon",cgpa:3.80},

  {name:"Rakib",cgpa:3.65}
]
```

## find()

---

### English

**find()** returns the **first matching element**.

### বাংলা

**find()** Condition অনুযায়ী প্রথম যে Element পাওয়া যায় সেটি Return করে।

এটি Array Return করে না।

একটি Object বা একটি Value Return করে।

## Why Use find()?

---

যখন শুধু একটি Data দরকার হয়।

যেমন

- একটি User
- একটি Product
- একটি Order
- একটি Book

তখন **find()** ব্যবহার করা হয়।

## Syntax

---

```
array.find((item)=>{

  return condition;
```



```
});
```



## Example 1

---

```
const numbers = [5,10,15,20];

const result = numbers.find(num=>num>10);

console.log(result);
```

Output

```
15
```

Explanation

১৫ পাওয়ার পর

Loop বন্ধ হয়ে যায়।



## Example 2

---

Product Search

```
const products = [

  {

    id:1,

    name:"Laptop"

  },

  {

    id:2,
```

```
        name: "Mouse"

    },

    {

        id: 3,

        name: "Keyboard"

    }

];

const product = products.find(item => item.id === 2);

console.log(product);
```

Output

```
{

  id: 2,

  name: "Mouse"

}
```

## Line by Line Explanation

---

```
products.find()
```

Loop শুরু করবে।

```
item.id === 2
```

Condition Check করবে।

True হলে

সাথে সাথে Object Return করবে।

এরপর Loop বন্ধ।



## findIndex()

---

### English

**findIndex()** returns the index of the first matching element.

### বাংলা

**findIndex()**

Element Return করে না।

Element-এর Position Return করে।



## Example

---

```
const fruits = [  
  "Apple",  
  "Orange",  
  "Banana"  
];  
  
const index = fruits.findIndex(  
  fruit=>fruit==="Orange"  
);  
  
console.log(index);
```

Output

1

কারণ

Apple → Index 0

Orange → Index 1

Banana → Index 2



## React Example

---

ধরো API থেকে Product এসেছে।

```
const products = [  
  {  
    id:1,  
    name:"Laptop",  
    available:true  
  },  
  {  
    id:2,  
    name:"Mouse",  
    available:false  
  },  
  {  
    id:3,
```

```
      name: "Keyboard",  
  
      available: true  
    }  
  ];  
};
```

Available Product

```
function Products(){  
  
  const availableProducts =  
  
    products.filter(product=>product.available);  
  
  return(  
  
    <div>  
  
    {  
  
      availableProducts.map(product=>  
  
        <h2 key={product.id}>  
  
          {product.name}  
  
        </h2>  
  
      )  
  
    }  
  
    </div>  
  
  );  
  
}
```

Output

Laptop

Keyboard



## Real MERN Project Example

---

Blood Donation Project

Database

users

Need

Only Donors

```
const donors = users.filter(  
  
  user=>user.role==="donor"  
  
);
```

Admin Panel

Need

Only Volunteers

```
const volunteers = users.filter(  
  
  user=>user.role==="volunteer"
```

```
);
```

Travel Booking Project

Need

Find Vehicle by ID

```
const vehicle = vehicles.find(  
  
  item=>item._id===id  
  
);
```

এই Pattern প্রায় সব MERN Project-এ ব্যবহার হয়।



## Common Mistakes

---

### filter() এর জায়গায় find()

Wrong

```
const user = users.filter(  
  
  item=>item.id===1  
  
);
```

এটি Array Return করবে।

যদি একটি User লাগে,

তাহলে

```
find()
```

ব্যবহার করো।

## find() এর জায়গায় map()

Wrong

```
products.map(item=>item.id===2);
```

map()

Search করার জন্য নয়।

Transform করার জন্য।

## findIndex() দিয়ে Object পাওয়ার চেষ্টা

Wrong

```
const user = users.findIndex(  
  item=>item.id===1  
);  
  
console.log(user.name);
```

findIndex()

Index Return করে।

Object নয়।

## Best Practices

---

- Search করার জন্য `find()`



- Filter করার জন্য `filter()`
- Position জানার জন্য `findIndex()`
- React-এ Filter + `map()` Combination ব্যবহার করো।
- Original Array পরিবর্তন করো না।

## Interview Questions

---

**filter() এবং find() এর পার্থক্য কী?**

**filter() কি Original Array পরিবর্তন করে?**

**find() কী Return করে?**

**findIndex() কী Return করে?**

**React-এ filter() কেন বেশি ব্যবহার হয়?**

**কোন ক্ষেত্রে filter() ব্যবহার করবে?**

**কোন ক্ষেত্রে find() ব্যবহার করবে?**



## Practice

---

### Beginner

একটি Number Array তৈরি করো।

10

15

20

25

30

শুধু

20+

Filter করো।

## Intermediate

একটি Student Array তৈরি করো।

শুধু

CGPA >= 3.50

Student বের করো।

## Advanced

একটি Product Array তৈরি করো।

Properties

id

name

price

available

Tasks

- Available Products Filter করো।
- ID দিয়ে Product Find করো।
- Laptop-এর Index বের করো।

## Summary

---

এই Part-এ তুমি শিখলে

- filter()
- find()
- findIndex()
- React Filter Pattern
- MERN Project Example
- Common Mistakes
- Best Practices

React Project-এ **filter()** এবং **find()** API Data নিয়ে কাজ করার সময় সবচেয়ে বেশি ব্যবহৃত Array Methods-এর মধ্যে দুটি।

## ES6 Arrow Function (Part 3B-2A)

---

*Beginner → Advanced Guide for Software Development*

## Table of Contents

- some()
- Why Use some()?
- Syntax
- Examples
- Line by Line Explanation
- every()
- Why Use every()?
- Syntax

- Examples
- React Examples
- MERN Project Examples
- Common Mistakes
- Best Practices
- Interview Questions
- Practice
- Summary

## some()

---

### English

The **some()** method checks whether **at least one element** in an array satisfies a condition.

It always returns a Boolean value.

```
true
```

or

```
false
```

### বাংলা

**some()** এমন একটি Array Method যা দেখে Array-এর **কমপক্ষে একটি Element** নির্দিষ্ট Condition পূরণ করেছে কি না।

যদি একটি Element-ও Condition পূরণ করে,

Result হবে

```
true
```

না করলে

```
false
```

সহজ ভাষায়,

ধরো,

একটি Exam-এ ৫০ জন Student আছে।

তুমি জানতে চাও,

"কেউ কি GPA 5.00 পেয়েছে?"

সব Student-এর Result দরকার নেই।

শুধু জানতে হবে,

কমপক্ষে একজন আছে কিনা।

এই কাজের জন্য

```
some()
```

ব্যবহার করা হয়।



## Why Use some()?

---

Real Project-এ

some()

অনেক ব্যবহার হয়।

যেমন

- কোনো User Logged In আছে?
- কোনো Product Out Of Stock?

- কোনো Student Failed?
- কোনো Order Pending?
- কোনো Notification Unread?

## Syntax

---

```
array.some((item)=>{  
    return condition;  
});
```

Short Syntax

```
array.some(item=>condition);
```



## Example 1

---

```
const numbers = [10,20,30,40];  
  
const result = numbers.some(  
    number=>number>25  
);  
  
console.log(result);
```

Output

```
true
```

কারণ

30

Condition পূরণ করেছে।



## Example 2

```
const numbers = [2,4,6,8];

const result = numbers.some(
  num=>num%2!==0
);

console.log(result);
```

Output

false

কারণ

কোনো Odd Number নেই।



## Line by Line Explanation

```
numbers.some()
```

Array Loop করবে।

```
number>25
```

Condition Check করবে।

প্রথম True পেলেই

Loop বন্ধ।

Return করবে

```
true
```

## every()

---

### English

The

```
every()
```

method checks whether **all elements** satisfy a condition.

If every element passes,

Result

```
true
```

Otherwise

```
false
```

### বাংলা



```
every()
```

দেখে Array-এর **সবগুলো Element** Condition পূরণ করেছে কি না।

একটিও যদি Fail করে,

Result

```
false
```

সহজ ভাষায়,

ধরো,

একটি Company-তে সব Employee-এর বয়স ১৮-এর বেশি হতে হবে।

একজনের বয়স ১৭ হলেই

Condition Fail

এটাই

```
every()
```

## Why Use every()?

ব্যবহার করা হয়

- Age Validation
- Form Validation
- Password Check
- Permission Check
- Payment Verification

## Syntax

```
array.every(item=>condition);
```



## Example 1

---

```
const numbers = [2,4,6,8];

const result = numbers.every(

num=>num%2===0

);

console.log(result);
```

Output

```
true
```

কারণ

সবগুলো Even Number।



## Example 2

---

```
const ages = [20,25,18,17];

const adult = ages.every(

age=>age>=18

);

console.log(adult);
```

Output

```
false
```

কারণ

```
17
```

Condition পূরণ করেনি।

## Line by Line Explanation

---

```
ages.every()
```

সবগুলো Element Check করবে।

```
age>=18
```

Condition I

যদি

একটিও False হয়,

Loop বন্ধ।

Result

```
false
```

## React Example

---

ধরো,

API থেকে Product এসেছে।

```
const products = [  
  
  {  
  
    id:1,  
  
    name:"Laptop",  
  
    stock:10  
  
  },  
  
  {  
  
    id:2,  
  
    name:"Mouse",  
  
    stock:0  
  
  },  
  
  {  
  
    id:3,  
  
    name:"Keyboard",  
  
    stock:15  
  
  }  
  
];
```

Check

কোনো Product Out Of Stock আছে?

```
const outOfStock = products.some(  
  
  product=>product.stock===0
```

```
);  
  
console.log(outOfStock);
```

Output

```
true
```

কারণ

Mouse-এর Stock

```
0
```



## React Example 2

---

সব Product Available?

```
const available = products.every(  
  product=>product.stock>0  
);
```

Output

```
false
```

কারণ

একটি Product-এর Stock নেই।



## MERN Project Example 1

---

Blood Donation Project

সব Donor কি Verified?

```
const verified = users.every(  
  
  user=>user.status==="verified"  
  
);
```

যদি

```
true
```

হয়

Admin Donation Accept করবে।



## MERN Project Example 2

---

Travel Booking Website

কোনো Booking Pending?

```
const pending = bookings.some(  
  
  booking=>booking.status==="pending"  
  
);
```

যদি

```
true
```

হয়

Dashboard Notification দেখাবে।



## MERN Project Example 3

---

E-Commerce

সব Product কি Published?

```
const published = products.every(  
  product=>product.publish===true  
);
```



## Common Mistakes

---

### Using filter()

Wrong

```
users.filter(  
  user=>user.status==="verified"  
);
```

যদি শুধু জানতে চাও

Verified User আছে কিনা,

তাহলে

```
some()
```

ব্যবহার করো।

## Using some()

Wrong

```
ages.some(  
  age=>age>=18  
);
```

যদি জানতে চাও

সবাই Adult কিনা,

তাহলে

```
every()
```

ব্যবহার করো।

## some() vs every()

| some() | every() | ||-| | কমপক্ষে একটি True হলেই True | সবগুলো True হতে হবে | | Boolean Return করে | Boolean Return করে | | প্রথম True পেলেই Loop Stop | প্রথম False পেলেই Loop Stop | | Exists Check | Validation Check |

## Best Practices

- কোনো একটি Data আছে কিনা → **some()**
- সবগুলো Data Valid কিনা → **every()**
- Validation-এর জন্য **every()**
- Existence Check-এর জন্য **some()**
- React Dashboard-এ Notification Check করতে **some()** ব্যবহার করো।



## Interview Questions



**some() কী Return করে?**

**every() কী Return করে?**

**some() এবং filter() এর পার্থক্য কী?**

**some() এবং every() এর পার্থক্য কী?**

**কোন ক্ষেত্রে some() ব্যবহার করবে?**

**কোন ক্ষেত্রে every() ব্যবহার করবে?**

**some() কি Original Array পরিবর্তন করে?**



## Practice

---

### Beginner

একটি Number Array তৈরি করো।

10

15

20

25

Check করো

কোনো Number

20+

আছে কিনা।

## Intermediate

একটি Student Array তৈরি করো।

Check করো

সব Student-এর

```
CGPA >= 3.00
```

কিনা।

## Advanced

একটি Product Array তৈরি করো।

Properties

```
id  
  
name  
  
stock  
  
price
```

Tasks

- কোনো Product Out Of Stock আছে কিনা।
- সব Product-এর Stock 0-এর বেশি কিনা।



## Summary

---

এই Part-এ তুমি শিখলে

- some()
- every()
- Boolean Return

- Validation Pattern
- React Example
- MERN Project Example
- Common Mistakes
- Best Practices

Software Development-এ

```
some()
```

এবং

```
every()
```

মূলত Validation, Permission Check, Notification System, Admin Dashboard এবং Business Logic তৈরিতে ব্যাপকভাবে ব্যবহৃত হয়।



## ES6 Arrow Function (Part 3B-2B-1)

*Beginner → Advanced Guide for Software Development*



### Table of Contents

- What is reduce()?
- Why Learn reduce()?
- Theory
- Syntax
- Understanding Accumulator
- Understanding Current Value
- Basic Examples
- Sum Example

- Maximum Value
- Minimum Value
- Shopping Cart Example
- Line by Line Explanation
- Common Mistakes
- Best Practices
- Interview Questions
- Practice
- Summary

## What is reduce()?

---

### English

The **reduce()** method reduces an entire array into a **single value**.

That single value can be:

- Number
- String
- Object
- Array
- Boolean

Unlike **map()** or **filter()**, **reduce()** does not return another transformed list by default. It combines all elements into one final result.

### বাংলা

**reduce()** হলো JavaScript-এর সবচেয়ে শক্তিশালী Array Method।

এটি পুরো Array-কে ধাপে ধাপে Process করে **একটি Final Value** তৈরি করে।

এই Final Value হতে পারে

- Number
- String

- Object
- Array
- Boolean

সহজ ভাষায়,

ধরো তোমার কাছে ১০টি Product-এর দাম আছে।

500

700

1000

300

তুমি জানতে চাও

Total Price

একটি সংখ্যা।

এখানেই

reduce()

সবচেয়ে ভালো Solution।

## Why Learn reduce()?

Software Development-এ reduce() অনেক জায়গায় ব্যবহার হয়।

যেমন

- Shopping Cart Total
- Dashboard Analytics
- Expense Tracker

- Sales Report
- Invoice Total
- Total Donation
- Total Booking Amount
- Student Average
- Category Count
- Financial Report

MERN Project-এ reduce() খুবই গুরুত্বপূর্ণ।

## Theory

---

ধরো,

তোমার কাছে একটি Bucket আছে।

এক এক করে সবাই সেখানে টাকা রাখছে।

100

↓

200

↓

300

↓

400

শেষে Bucket-এ

1000

টাকা হলো।

Bucket-টাই হচ্ছে

Accumulator

আর

100

200

300

400

হলো Current Value।

প্রতিবার নতুন Value Bucket-এর সাথে যোগ হচ্ছে।

এটাই reduce()।

## Syntax

---

```
array.reduce((accumulator, currentValue) => {  
    return updatedValue;  
}, initialValue);
```

Short Syntax

```
array.reduce(  
    (acc, current) => acc + current,  
    0  
);
```



## Understanding Accumulator

---

Accumulator হলো এমন একটি Variable

যেখানে Previous Result জমা থাকে।

Example

```
const numbers = [10,20,30];
```

Iteration

Step 1

acc = 0

current = 10

Result = 10

Step 2

acc = 10

current = 20

Result = 30

Step 3

acc = 30

current = 30

Result = 60

Final

60





## Understanding Current Value

---

Current Value হলো

বর্তমানে যে Element-এর উপর Loop চলছে।

Example

```
const fruits = [  
  "Apple",  
  "Orange",  
  "Banana"  
];
```

Iteration

```
Apple  
↓  
Orange  
↓  
Banana
```

প্রতিটি Element একবার করে

Current Value হবে।



### Example 1

---

সব Number যোগ করো।

```
const numbers = [10,20,30,40];
```

```
const total = numbers.reduce(  
  
  (acc,current)=>{  
  
    return acc+current;  
  
  },  
  
  0  
  
);  
  
console.log(total);
```

Output

100

## Line by Line Explanation

---

`numbers.reduce()`

Array-এর প্রতিটি Element-এর উপর Loop করবে।

`(acc,current)`

acc

↓

Previous Result

current

↓

Current Number

```
return acc+current;
```

দুইটিকে যোগ করবে।

```
0
```

Initial Value।

প্রথমে

```
acc=0
```

থাকবে।



## Iteration Table

| Step | acc | current | Result |
|------|-----|---------|--------|
| 1    | 0   | 10      | 10     |
| 2    | 10  | 20      | 30     |
| 3    | 30  | 30      | 60     |
| 4    | 60  | 40      | 100    |

Final Result

```
100
```



## Example 2

সব Price যোগ করো।

```
const prices = [  
  
500,  
  
800,  
  
1200,
```

```
400

];

const total = prices.reduce(

(acc,price)=>acc+price,

0

);

console.log(total);
```

Output

```
2900
```



## Real Project Example

---

Shopping Cart

```
const cart = [

{

name:"Laptop",

price:50000

},

{

name:"Mouse",

price:1200

},

{
```

```
name:"Keyboard",  
  
price:2500  
  
}  
  
];
```

Total Price

```
const total = cart.reduce(  
  
  (acc,item)=>{  
  
    return acc+item.price;  
  
  },  
  
  0  
  
);  
  
console.log(total);
```

Output

```
53700
```

এটি E-Commerce Website-এর সবচেয়ে Common Logic।



## Example 3

---

Maximum Number

```
const numbers = [  
  
  15,
```

```
20,  
80,  
35,  
10  
];  
  
const max = numbers.reduce(  
  (acc,current)=>{  
    return current>acc  
      ?current  
      :acc;  
  }  
);  
  
console.log(max);
```

Output

80

Explanation

প্রতিবার

Current Number

এবং

Previous Maximum

Compare করবে।



## Example 4

Minimum Number

```
const numbers = [

15,

20,

80,

35,

10

];

const min = numbers.reduce(

(acc,current)=>{

return current<acc

?current

:acc;

}

);

console.log(min);
```

Output

10

## Common Mistakes

---

### Forgetting Initial Value

Wrong

```
numbers.reduce(  
  
  (acc,current)=>acc+current  
  
);
```

এটি অনেক ক্ষেত্রে Bug তৈরি করতে পারে।

Best Practice

```
numbers.reduce(  
  
  (acc,current)=>acc+current,  
  
  0  
  
);
```

## Confusing map() with reduce()

Wrong

```
numbers.map(  
  
  num=>num+10  
  
);
```

এটি Total বের করবে না।

Total বের করতে

```
reduce()
```

ব্যবহার করতে হবে।



## Best Practices

---

- সবসময় Initial Value দাও।
- Meaningful Variable Name ব্যবহার করো।
- Complex Logic হলে Curly Braces ব্যবহার করো।
- Shopping Cart-এর Total বের করতে reduce() ব্যবহার করো।
- Dashboard-এর Total Analytics-এর জন্য reduce() ব্যবহার করো।

## Interview Questions

---

**reduce() কী?**

**reduce() কী Return করে?**

**Accumulator কী?**

**Current Value কী?**

**Initial Value কেন ব্যবহার করা হয়?**

**reduce() এবং map() এর পার্থক্য কী?**

**reduce() কোথায় ব্যবহার করা হয়?**



## Practice

---

### Beginner

একটি Array তৈরি করো।

10

20

30

```
40
```

`reduce()`

ব্যবহার করে Sum বের করো।

## Intermediate

একটি Shopping Cart তৈরি করো।

```
price
```

```
quantity
```

Total Price বের করো।

## Advanced

একটি Number Array তৈরি করো।

```
15
```

```
22
```

```
80
```

```
33
```

```
10
```

`reduce()`

ব্যবহার করে

- Maximum বের করো।
- Minimum বের করো।

## Summary

---

এই Part-এ তুমি শিখলে

- `reduce()`
- Accumulator
- Current Value
- Initial Value
- Sum
- Maximum
- Minimum
- Shopping Cart Total
- Iteration Process

`reduce()` হলো JavaScript-এর সবচেয়ে শক্তিশালী Array Method। এটি Dashboard, Analytics, Shopping Cart, Finance, Reporting এবং MERN Stack Project-এ নিয়মিত ব্যবহৃত হয়। যদি `reduce()` ভালোভাবে আয়ত্ত্ব করতে পারো, তাহলে জটিল Data Processing অনেক সহজ হয়ে যাবে।

## ES6 Arrow Function (Part 3B-2B-2A)

---

*Beginner → Advanced Guide for Software Development*

## Table of Contents

- Expense Tracker
- Dashboard Analytics
- Sales Report
- Total Revenue
- Average Revenue
- React Example
- MERN Project Example
- Line by Line Explanation
- Common Mistakes

- Best Practices
- Interview Questions
- Practice
- Summary



## Expense Tracker

---

### English

An Expense Tracker records every expense and calculates the total amount spent.

In real applications, expenses are usually stored as objects inside an array.

### বাংলা

Expense Tracker এমন একটি Application যেখানে প্রতিটি খরচ (Expense) সংরক্ষণ করা হয় এবং শেষে মোট কত টাকা খরচ হয়েছে তা বের করা হয়।

এই কাজের জন্য JavaScript-এর সবচেয়ে জনপ্রিয় Method হলো

```
reduce()
```

কারণ এটি পুরো Array-এর Data যোগ করে একটি Final Result তৈরি করতে পারে।

Example Data

```
const expenses = [  
  {  
    title: "Food",  
    amount: 500  
  },  
  {  
    title: "Internet",
```

```
        amount: 800

    },

    {

        title: "Transport",

        amount: 300

    }

];
```



## Calculate Total Expense

---

```
const totalExpense = expenses.reduce(

    (acc, expense) => {

        return acc + expense.amount;

    },

    0

);

console.log(totalExpense);
```

Output

```
1600
```



## Line by Line Explanation

---

```
expenses.reduce()
```

Array-এর প্রতিটি Expense-এর উপর Loop করবে।

```
(acc, expense)
```

```
acc
```

↓

আগের Total

```
expense
```

↓

বর্তমান Expense Object

```
expense.amount
```

বর্তমান Expense-এর Amount

```
return acc + expense.amount;
```

Total Update করবে।

Initial Value

0

মানের শুরুতে Total = 0



## Iteration Table

| Step | Previous Total | Current Expense | New Total | ||:-



-



|1|0|500|500| |2|500|800|1300| |3|1300|300|1600|

Final Result

1600



## Dashboard Analytics

Dashboard-এ সাধারণত নিচের তথ্যগুলো দেখানো হয়।

- Total Users
- Total Orders
- Total Revenue
- Total Donation
- Total Bookings

এই ধরনের সব Calculation-এর জন্য

`reduce()`

সবচেয়ে বেশি ব্যবহার হয়।

Example

```
const orders = [  
  {  
    id:1,  
    total:500  
  },  
  {  
    id:2,  
    total:1200  
  },  
  {  
    id:3,  
    total:800  
  }  
];
```

Total Revenue

```
const revenue = orders.reduce(  
  (acc, order) => {  
    return acc + order.total;  
  },  
  0  
);
```



```
console.log(revenue);
```

Output

```
2500
```



## Sales Report

---

ধরো

একটি দোকানে প্রতিদিনের বিক্রির তথ্য আছে।

```
const sales = [  
  1000,  
  1500,  
  900,  
  700,  
  1200  
];
```

Total Sales

```
const totalSales = sales.reduce(  
  (acc, sale) => acc + sale,  
  0  
);  
  
console.log(totalSales);
```

Output

5300

## Average Revenue

---

Average বের করার জন্য

প্রথমে Total বের করতে হবে।

তারপর

Array-এর Length দিয়ে ভাগ করতে হবে।

```
const revenue = [1000,2000,3000];  
  
const total = revenue.reduce(  
  (acc,item)=>acc+item,  
  0  
);  
  
const average = total / revenue.length;  
  
console.log(average);
```

Output

2000

বাস্তব Dashboard-এ Average Revenue, Average Sales, Average Rating—সব ক্ষেত্রেই এই Logic ব্যবহার করা হয়।



## React Example

ধরো

API থেকে Cart Data এসেছে।

```
const cart = [

  {

    id:1,

    name:"Laptop",

    price:50000

  },

  {

    id:2,

    name:"Mouse",

    price:1200

  }

];
```

React Component

```
function Cart(){

  const total = cart.reduce(
```

```

    (acc,item)=>acc+item.price,

    0

  );

  return(

    <div>

      <h2>

        Total : {total}

      </h2>

    </div>

  );

}

```

Output

```
Total : 51200
```

এটি React-এর Shopping Cart-এর Standard Pattern।



## MERN Project Example

---

Blood Donation Project

Donations Collection

```

const donations = [

  {

    amount:1000

  },

```

```
{  
  
  amount:500  
  
},  
  
{  
  
  amount:250  
  
}  
  
];
```

Total Donation

```
const totalDonation = donations.reduce(  
  
  (acc,item)=>acc+item.amount,  
  
  0  
  
);  
  
console.log(totalDonation);
```

Output

1750

Admin Dashboard

Total Donation  
  
1750 BDT



## Travel Booking Project

---

## Bookings

```
const bookings = [  
  
  {  
  
    price:1200  
  
  },  
  
  {  
  
    price:1800  
  
  },  
  
  {  
  
    price:2500  
  
  }  
  
];
```

## Revenue

```
const revenue = bookings.reduce(  
  
  (acc,item)=>acc+item.price,  
  
  0  
  
);
```

## Dashboard

Total Booking Revenue

5500

## Common Mistakes

---

### Forgetting Initial Value

Wrong

```
expenses.reduce(  
  (acc,item)=>acc+item.amount  
);
```

Correct

```
expenses.reduce(  
  (acc,item)=>acc+item.amount,  
  0  
);
```

### Using map()

Wrong

```
expenses.map(  
  item=>item.amount  
);
```

এটি Amount-এর Array Return করবে।

Total নয়।

Total বের করতে

```
reduce()
```

ব্যবহার করতে হবে।

## Best Practices

---

- Total বের করার জন্য reduce() ব্যবহার করো।
- Initial Value সবসময় দাও।
- Meaningful Variable Name ব্যবহার করো।
- Dashboard Analytics-এ reduce() ব্যবহার করো।
- Shopping Cart Logic আলাদা Utility Function-এ রাখো।

Example

```
const calculateCartTotal = (cart) => {  
  
  return cart.reduce(  
  
    (acc,item)=>acc+item.price,  
  
    0  
  
  );  
  
};
```

এটি Reusable এবং Professional।

## Interview Questions

---

**Expense Tracker-এ reduce() কেন ব্যবহার করা হয়?**

**Dashboard Analytics-এ reduce() কোথায় ব্যবহার হয়?**



Average Revenue কীভাবে বের করবে?

map() এবং reduce() এর পার্থক্য কী?

Shopping Cart-এর Total কীভাবে বের করবে?

Initial Value কেন গুরুত্বপূর্ণ?



## Practice

---

### Beginner

একটি Expense Array তৈরি করো।

```
Food
```

```
Internet
```

```
Transport
```

Total Expense বের করো।

### Intermediate

একটি Order Array তৈরি করো।

```
id
```

```
total
```

Total Revenue বের করো।

Average Revenue বের করো।

### Advanced

একটি Shopping Cart তৈরি করো।

Properties

```
id  
  
name  
  
price  
  
quantity
```

Task

- Total Price
- Total Quantity
- Average Price

reduce()

ব্যবহার করে বের করো।



## Summary

---

এই Part-এ তুমি শিখলে

- Expense Tracker
- Dashboard Analytics
- Sales Report
- Total Revenue
- Average Revenue
- Shopping Cart Total
- React Example
- MERN Example
- Best Practices

Software Development-এ **reduce()** শুধু Array-এর Number যোগ করার জন্য নয়, বরং **Business Logic** তৈরির জন্য ব্যবহার করা হয়। Finance, E-Commerce, Analytics, Admin Dashboard, Booking System, Blood Donation Platform—প্রায় সব বড় Project-এ **reduce()** গুরুত্বপূর্ণ ভূমিকা পালন করে।



## ES6 Arrow Function (Part 3B-2B-2B)

*Advanced reduce() Examples for Software Development*



### Group Data by Category

ধরো তোমার কাছে Product List আছে।

```
const products = [
  { name: "Laptop", category: "Electronics" },
  { name: "Mouse", category: "Electronics" },
  { name: "Shirt", category: "Fashion" },
  { name: "Pant", category: "Fashion" },
];
```

Category অনুযায়ী Group করো।

```
const grouped = products.reduce((acc, product) => {
  if (!acc[product.category]) {
    acc[product.category] = [];
  }

  acc[product.category].push(product);

  return acc;
}, {});

console.log(grouped);
```

Output

```
{
  Electronics: [
    { name: "Laptop", category: "Electronics" },
    { name: "Mouse", category: "Electronics" }
  ],
}
```

```
Fashion: [  
  { name: "Shirt", category: "Fashion" },  
  { name: "Pant", category: "Fashion" }  
]  
}
```

ব্যবহার

- Product Category
- Student Department
- Blog Category
- News Category

## Count Objects by Category

---

```
const users = [  
  { role: "Admin" },  
  { role: "User" },  
  { role: "Admin" },  
  { role: "Moderator" },  
  { role: "User" }  
];
```

```
const count = users.reduce((acc, user) => {  
  
  acc[user.role] = (acc[user.role] || 0) + 1;  
  
  return acc;  
  
}, {});  
  
console.log(count);
```

Output

```
{  
  Admin: 2,
```

```
User: 2,  
Moderator: 1  
}
```

ব্যবহার

- Role Count
- Blood Group Count
- Vehicle Type Count
- Order Status Count



## Most Expensive Product

---

```
const products = [  
  
  { name: "Laptop", price: 50000 },  
  
  { name: "Phone", price: 30000 },  
  
  { name: "Mouse", price: 1200 }  
  
];
```

```
const expensive = products.reduce((max, product) => {  
  
  return product.price > max.price  
  
    ? product  
  
    : max;  
  
});  
  
console.log(expensive);
```

Output

```
{  
  name: "Laptop",  
  price: 50000  
}
```



## MERN Admin Dashboard Example

---

Dashboard Card

```
Total Users  
  
Total Products  
  
Total Orders  
  
Total Revenue  
  
Pending Orders
```

Example

```
const revenue = orders.reduce(  
  
  (acc, order) => acc + order.total,  
  
  0  
  
);  
  
const pending = orders.filter(  
  
  order => order.status === "Pending"  
  
).length;
```

React

```
<h2>Total Revenue : {revenue}</h2>

<h2>Pending Orders : {pending}</h2>
```

## Professional Project Uses

---

Shopping Cart

Expense Tracker

Sales Dashboard

Blood Donation Dashboard

Student Result System

Travel Booking

Admin Panel

Analytics Report

E-commerce

## Common Mistakes

---

map() দিয়ে Total বের করা

```
products.map(item => item.price);
```

✓ Correct

```
products.reduce(

(acc, item) => acc + item.price,

0

);
```

filter() দিয়ে একটি Object খোঁজা

✓ Correct

```
products.find(  
  
  item => item.id === 1  
  
);
```



## Best Practices

---

- map() → Transform Data
- filter() → Filter Data
- find() → One Item
- some() → At Least One
- every() → All Items
- reduce() → One Final Value



## Interview Questions

---

**map() vs filter()**

**filter() vs find()**

**some() vs every()**

**reduce() কী?**

**Accumulator কী?**

**reduce() কোথায় ব্যবহার হয়?**

**কেন React-এ map() বেশি ব্যবহার হয়?**



**Shopping Cart-এর Total কীভাবে বের করবে?**

**Dashboard Revenue কীভাবে বের করবে?**

**Group By কীভাবে করবে?**



## **Final Practice**

---

### **Project 1**

Student Management

- Total Students
- Passed Students
- Failed Students
- Average GPA

### **Project 2**

Shopping Cart

- Total Price
- Total Quantity
- Highest Price
- Cheapest Product

### **Project 3**

Blood Donation

- Total Donors
- Total Requests
- Pending Requests
- Approved Requests
- Total Donations

## Project 4

### Travel Booking

- Total Vehicles
- Available Vehicles
- Total Revenue
- Pending Booking
- Completed Booking



## ES6 Array Methods Cheat Sheet

| Method      | Return       | Use Case                 |
|-------------|--------------|--------------------------|
| map()       | New Array    | Transform Data           |
| filter()    | New Array    | Filter Data              |
| find()      | One Element  | Search One Item          |
| findIndex() | Index        | Get Position             |
| some()      | Boolean      | At Least One             |
| every()     | Boolean      | Validate All             |
| forEach()   | Undefined    | Loop Only                |
| reduce()    | Single Value | Total, Analytics, Report |



## Final Summary

React এবং MERN Project-এ সবচেয়ে বেশি ব্যবহৃত Array Methods হলো:

- map()
- filter()
- find()
- some()
- every()
- reduce()

এই Method-গুলো ভালোভাবে আয়ত্ত করতে পারলে তুমি Shopping Cart, Dashboard, Admin Panel, Analytics, Report, Booking System, Blood Donation Platform এবং বড় বড় MERN Project-এর ৮০% Data Processing Logic সহজেই লিখতে পারবে।